



A Tool for Moving Blockchain Computations Off-Chain

Christian Liu
Faculty of Computer Science
Dalhousie University
Halifax, Nova Scotia, Canada
Chris.Liu@dal.ca

Peter Bodorik
Faculty of Computer Science
Dalhousie University
Halifax, Nova Scotia, Canada
Peter.Bodorik@dal.ca

Dawn Jutla
Sobey School of Business
Saint Mary's University
Halifax, Nova Scotia, Canada
Dawn.Jutla@smu

ABSTRACT

This paper proposes a new approach and tool for blockchain software developers and architects to use for determining which computations of a smart contract can be effectively done off-chain without loss of trust and how they can be moved off-chain automatically. Our approach uses Finite State Machines (FSMs) and Hierarchical State Machines (HSMs) modeling in order to create smart contract patterns using graphs and then use pattern properties to identify which parts of the smart contracts should be considered for moving off-chain. Once software execution transits into the pattern's entry state, off-chain execution continues until the execution transits into the pattern's exit state, when execution continues on-chain. This paper further describes a tool for (1) FSM input and visualization, its analysis, and automatic transformation into a smart contract, and (2) an interface to support the developers' decision to optimally off-chain computations and to (3) define the communications between on-chain and off-chain computations.

CCS CONCEPTS

• Computing methodologies → Distributed computing methodologies • Networks → Network services • Computer systems organization → Architectures • Software and its engineering → Software organization and properties • Information systems → Information systems applications • Computing methodologies → Modeling and simulation

KEYWORDS

Blockchain; smart contract; Off-chain computation; FSM model; Hierarchical State Machine; HSM; simple-graph; transformation of FSM to smart contract.

ACM Reference format:

Christian G. Liu, Peter Bodorik, Dawn Jutla. 2021. A Tool for Moving Blockchain Computations Off-Chain. In *Proceedings of 2021 3rd ACM*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

BSCI'21, June 3–7, 2021, Virtual Event, Hong Kong.

© 2021 Copyright is held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8400-1/21/06...\$15.00.
<https://doi.org/10.1145/3457337.3457848>

International Symposium on Blockchain and Secure Critical Infrastructure (BSCI'21), June 3–7, 2021, Virtual Event, Hong Kong. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3457337.3457848>

1 Introduction

One of the approaches, adopted by researchers and practitioners in handling the issues the blockchain scalability is to create models and techniques for moving computation and storage off the blockchain. Eberhardt and Tai [2] categorized methods, for reducing the blockchain size, based on what is moved to off-chain into: (i) off-chain storage, (ii) off-chain computation, and (iii) a hybrid approach, in which both storage and computation are off-chained. In this paper, we use the term off-chaining to refer to moving both computation and storage off the chain where it is performed using some other mechanisms, such as sidechains.

Off-chain storage is a well-established method in which data is stored off-chain in streams to which nodes subscribe and has been adopted, for instance, in [5]. All off-chain storage methods store an object off-chain but also store the object's hash code on the blockchain so that it can be used to ascertain its object's immutability any time that object is retrieved. We also assume that all storage of objects off-chain uses this mechanism.

Poon and Buterin [10] describe the Plasma project in which computation is performed off-chain while the blockchain is used for recording of certifications that off-chain computation is correct. How to achieve attestation of results produced by off-chain processing as being correct is not described.

1.1 Our Approach

A smart contract may be considered as a collection of methods used to automate or transform use scenarios in a blockchain application. When we refer to a pattern to be processed off-chain, we are referring to a collection of methods and supporting data to be processed off-chain. We break down the problem of processing off-chains into three relatively independent subproblems, to which we broadly speaking refer to briefly as “What”, “When”, i.e., under which conditions, and “How”, respectively:

1. The first subproblem deals with determining which pattern should be processed off-chain.
2. For a pattern to be processed off-chain, (a) overhead cost of processing off-chain should be lower than benefits of processing off-chain, and (b) processing off-chain should not diminish the beneficial blockchain properties, such as immutability of blockchain data or trust.

3. The final issue is how to facilitate interfaces automatically for interaction between on and off-chain processing, a main contribution of this paper.

1.2 Objectives

Our main objectives of this paper are two-fold:

1. Describe our approach in solving the three issues of “What”, “When”, and “How” and
2. Describe a tool that we build to assist a developer in solving the above issues.

It should be noted that we are targeting applications in Trade Finance that may have complex interactions amongst various parties but are also long running.

1.3 Outline

Section 2 provides background research results that we use. Sections 3 and 4 describe our approach to finding patterns to be processed off-chain and our progress on the second issue of “When” should a pattern be processed off-chain. Section 5 describes how the FSM model is transformed into a smart contract that it also includes the interface for interaction between on and off-chain processing. Section 6 describes our tool to assist a designer of a smart contract with all three issues. The final section provides a summary and conclusions.

2 Background

We briefly review Finite State Machines (FSMs) and also Hierarchical State Machines (HSMs) that are used to model blockchains. As we also use both concepts here, we briefly review their notation. Following this we describe modeling and use of FSMs for ensuring security of smart contracts [8, 9].

FSMs have been used for a long time for modeling of transactions and applications in many domains, including business and commerce. As smart contracts execution on blockchains includes state data/variables that are stored on the blockchain, they are suitable for modeling of smart contracts using FSM. As we also use FSMs to model blockchain smart contracts, we briefly review the FSM notation adopted from [8] with the exception of guards and labels that we omit for simplicity in presentation: An FSM F can be described as $F = (S, s_0, T, I, O)$, where:

- S : Set of n states $\{s_0, s_1, \dots, s_{n-1}\}$, where s_0 is the initial state of the FSM
- T : Set of transitions $\{t_i, i=1, 2, \dots, m\}$, such that each t_i has two components, $t_i.\text{from}$ and $t_i.\text{to}$, that identify the transition to be from the state $t_i.\text{from}$ to the state $t_i.\text{to}$, respectively
- I : Set of inputs $\{x_i\}$. Each transition t_i has input x_i that causes the transition. (x_i may be a large object or a set of objects)
- O : Set of outputs $\{y_j\}$. Each transition t_j has a set of outputs where each y_j may be a single value or it may be an object or a set of objects, whose value is produced by the state activities and is output by the state transition.

We note that S is a connected graph and, furthermore, with the exception of the start and final nodes, we assume that each node

has at least one incoming and at least one outgoing edge (otherwise the FSM is not considered to be well-formed).

In late 80's, FSMs have been extended with the concept of hierarchy, leading to HSMs that can contain states that are themselves other FSMs. As we also utilize the concept of HSM, we review it briefly. An HSM can be defined using induction as follows [11]. In the base case, an FSM is a hierarchical machine. Suppose that M is a set of HSMs. If F is an FSM with a set of states, S , and there is a mapping function $f: S \rightarrow M$, then the triple (F, M, f) is an HSM. Any HSM has a corresponding equivalent FSM that can be achieved by “flattening” the hierarchy in HSM: Each state, $s \in S$, that represents an HSM is replaced by its mapping $(f(s))$. HSMs recognize the same language as its corresponding flattened FSM. HSMs do not increase expressiveness of FSMs, only succinctness in representing them.

Our work draws upon research results described in [8, 9], on which FSM modeling is used to represent the smart and then transforming the FSM model into methods of a smart contract in order to address its security issues. We also use FSM modeling to represent the smart contract that we transform to smart contract methods together with interface for off-chain processing.

We note that patterns in software and smart contracts have played a major role in the design and implementation of blockchain smart contracts. Examples of such patterns were identified in [3], which include Fail early and Fail loud, State machine, Upgradable registry, Transition counter and other patterns, and a Blind-bidding pattern in [8]. They also include patterns used for mitigating various security issues, such as a locking pattern to prevent re-entrance and access-control pattern in programming of smart contracts [9].

3 What to Process Off-chain

We model a smart contract using an FSM $F = (S, s_0, T, I, O)$ and use the FSM to represent a smart contract as a graph represented by the set of states, S , as being nodes, and transitions T , as a set of directed edges [8]. A pattern in an FSM graph that is represented by a subgraph S' , $S' \subseteq S$, that contains a set of states, S' , and directed edges representing state transitions. After trying a number of approaches [6], we determined that our search should be for a pattern that represents coordination of activities of actors that lead to one common goal. Further investigation has revealed that such a pattern forms a subgraph, of the FSM graph, with special properties in that all nodes, with the exception of one *entry node* and one *exit node*, are such that they have only internal edges, where an internal edge is between nodes of the subgraph itself. An entry node may have incoming edges from nodes outside the subgraph in addition to internal edges to other nodes of the subgraph (but it may not have outgoing external edges). Similarly, an exit node may have outgoing edges to nodes that are not members of the subgraph. We call such subgraphs as *simple subgraphs*. A simple subgraph is suitable for off-chaining as once it is entered by a transition into the entry node, its processing is “internal” to the subgraph until a transition occurs to the subgraph's exit node.

Furthermore, we also showed that, if a simple subgraph, say S , contains other simple subgraphs, such as S' and S'' , then all nodes of S' and S'' are also contained in S , and that S' and S'' are (i) either mutually exclusive or (ii) share at most one node that is entry (exit) node of S' and exit (entry) node of S'' . Furthermore, we also showed that, if a simple subgraph, say S , contains other simple subgraphs, such as S' and S'' , then all nodes of S' and S'' are also contained in S , and that S' and S'' are (i) either mutually exclusive or (ii) share at most one node that is entry (exit) node of S' and exit (entry) node of S'' .

4 When to Process Off-chain

The previous section described an algorithm to find simple subgraph that are good candidates for processing off-chain. In this section we describe our progress on the decision making when, that is under which condition, should a simple subgraph be processed off-chain. We first describe our method of presenting the simple subgraph patterns to the developer for deciding if a pattern should be processed off-chain and, if so, adaptation of the FSM graph to represent the processing off-chain. We then discuss the two main issues that affect the decision: (i) overhead cost vs benefits and (ii) trust issues due to processing off-chain. However, before we proceed, we first describe a Trade-finance use case we use as an example in applying our methods.

Consider an example that involves a seller and a buyer of some product that includes an escrow deposit [1]. After the product is posted for sale, buyer and seller negotiate price. Once an agreement is reached on the price, contract is prepared and signed that stipulates escrow deposit and the matter of delivery. Once each of the buyer and seller makes a deposit to an escrow account, then shipment, which includes crossing borders and hence involving customs, may occur that involves delivery to a port, going through customs, storing on a ship, then processed at the destination customs, unload from ship to port, buyer pickup, and finally return of escrow deposit. We created an FSM for this case and then applied our algorithm to find all simple subgraphs. Figure 1 shows the graph of the FSM model and also simple subgraphs that were identified by our algorithm. Note that simple subgraphs may contain subsets of the FSM nodes that are also simple subgraphs. For instance, in Figure 1, the simple subgraph S_6 contains two simple subgraphs S_2 and S_3 , where S_2 and S_3 are proper subsets of S_6 . Recall also that two simple subgraphs, are such that either (i) one is a subset of the other or (ii) they share at most one node that, furthermore, must be an entry node for one of the subgraphs and an exit node for the other one. In Figure 1, for instance, the simple subgraphs S_3 and S_4 share the node "Escrow Deposits Done". For formal presentations of these properties and algorithm to find simple subgraphs, see [6].

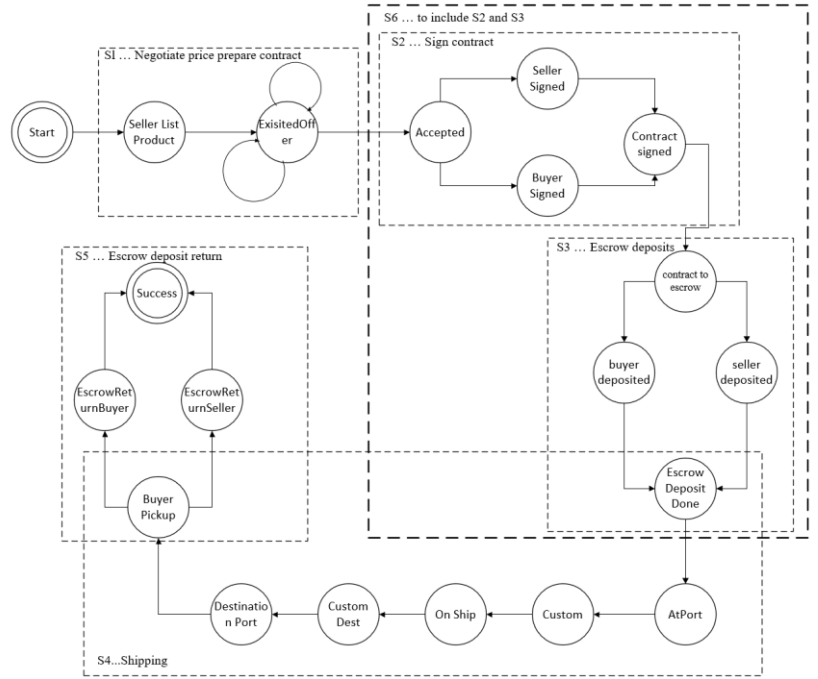


Figure 1: FSM graph for buyer-seller with escrow account

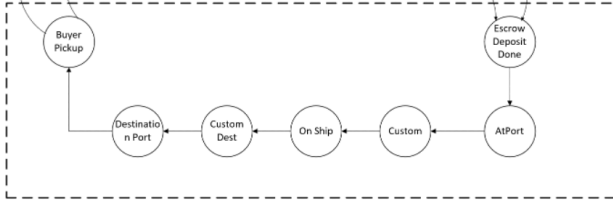
There are additional simple subgraphs that were found that we did not show in the figure as it would get too busy.

Concluding this section, we note that we applied our algorithm to find simple subgraphs for many patterns, such as: Fail early and Fail loud, State machine, Upgradable registry, Transition counter and other patterns in [3]; letter of credit [7], Blind-bidding pattern in [8]; and patterns used for mitigating various security issues, such as a locking pattern to prevent re-entrance and access-control pattern in programming of smart contracts [8, 9]. However, the limitation of our work is that, until we do exhaustive use case testing and validation, we cannot and do not claim that our algorithm finds all universal patterns that may be suitable to be processed off-chain.

4.1 Processing Simple Subgraphs

Simple subgraphs, together with relevant information, are presented to the developer in an iterative fashion, one at a time, in order for the developer to decide whether or not the pattern should be executed off-chain. If the developer decides not to off-chain a presented subgraph, then next simple subgraph is presented to the developer for consideration. However, if the developer does decide that the subgraph should be processed off-chain, then the simple subgraph is added to the output-list of simple subgraphs that are to be considered for processing off-chain. Furthermore, the FSM F is amended to reflect the developer's decision that the simple subgraph is to be processed off-chain – for that purpose we use HSMs and we modify FSM F into HSM H , by replacing the simple subgraph with an HSM node that represents the FSM of the simple-subgraph moved off-

chain. Consider, for example, the simple subgraph S4 that represents the product shipment in Figure 1 and that is repeated in Figure 2(a). If the subgraph should be processed off-chain, then the FSM graph is modified as shown in Figure 2(b) by replacing the pattern S4 in the FSM graph with an HSM node.



(a) Simple subgraph before decision to off-chain



(b) Simple subgraph after decision to off-chain

Figure 2: Escrow deposit simple subgraph before and after off-chain decision

Another consideration is a case when a pattern, such as S6 in Figure 1, contains other patterns, such as S3 and S4, some of which or both were already slated to be processed off-chain. When the developer considers the pattern S6 for off-chain processing, unless she states otherwise, S3 and S4 are incorporated as parts of processing the pattern S6. It is for that reason that the simple subgraphs are presented to the developer for decision making in sorted descending order as defined by the number of other simple subgraphs that a subgraph has.

4.2 Overhead Cost vs Benefits

We adopted an approach of Marvidou & Lazska [9] for transforming an FSM into a smart contract, which is a collection of smart contract methods. However, at the same time as the FSM is being transformed, we also prepare automatically an interface for the interaction of the smart contract, executing on the blockchain, with the methods of a pattern that is to be processed off-chain. We developed such software, and it enables us to actually run the methods of the smart contract and thus measure the cost of each method when there is no off-chain processing and when a pattern is processed off-chain, which also includes the on-chain cost of interface interaction with off-chain processing. We do this analysis for each method and then we ask the developer to provide information on the expected frequency of invocation of each smart contract method to summarize over all methods of a smart contract and thus provide to the developer information on the overhead costs due to off-chain processing. In the next section we describe how we transform an FSM into a smart contract and prepare automatically an interface between on and off-chain computation.

4.3 Trust Issues

Processing of parts of a smart contract off-chain should not affect negatively the main desirable properties of blockchain and their smart contracts, namely the properties of immutability, availability, and trust. We discuss these properties and how techniques used to maintain these properties when using off-chain storage and processing.

Immutability: When off-chain storage is used, immutability is safeguarded by all blockchains using hashes. Object or data stored off-chain is signed with a hash-code that is stored on the blockchain. When data stored off-chain is retrieved, its hash-code is recalculated and compared to the one stored on the blockchain to ensure that the data retrieved off-chain has not been changed. This approach is universally used for all off-chain data storage.

Availability: As a system availability is determined by the availability of its components, if computation is performed off-chain, its availability will affect the overall system availability. Hence, off-chain computation also needs to support availability. Fortunately, providing a desired availability via replication is a well-established and researched problem with good solutions.

Trust: This is the most difficult issue. Trust is gained by blockchains and their smart contracts because all parties can examine the smart contract code and changes to the state variables (in permissioned blockchains, as long access rights are available). As parts of the smart contract are moved off-chain, that may decrease trust on the part of the blockchain participants as questions may be raised regarding the off-chain processing and its implication on trust. The main approach to mitigating this risk is to use attestation of results produced by off-chain computation: After the off-chain processing is completed, the results of off-chain processing are provided to participants for attestation that they are correct by the participants digitally signing the results. In fact, we did not find in literature a solution to the general problem of how to attest results of off-chain computation. Although currently we too cannot provide a solution to this problem, we do collect all information available about a pattern in order to be presented to the developer for informed decision-making.

5 How: From FSM to Off-chain Interface

As in [9], we also use FSM modeling, which is augmented also with the use of HSMs, to transform our HSM model into a set of smart contract methods. Following this, however, for any pattern that is to be processed off-chain, we develop interface for interacting with the off-chain computation.

Due to the properties of simple subgraphs, once there is a transition into the entry node/state of the subgraph pattern that is processed off-chain, computation is entirely off-chain within the subgraph pattern until there is a transition out of the pattern (pattern's exit node), after which computation proceeds on the main chain. As the smart contract methods read from and write to the blockchain, off-chain execution of smart contract methods also requires reading from and writing to the blockchain; however, off-chain execution cannot write data to the blockchain

until off-chain computation is completed and results to be written to the blockchain are attested as correct. Consequently, writes to the blockchain by off-chain methods are recorded in a local cache that is prepared upon transition to off-chain processing.

Each smart contract method has parameters, which are in HSM representation are inputs to a transition, and it returns some values/object that correspond to the transition outputs. Thus, each method invocation, including invocation of a method processed off-chain, must include them.

Upon moving back to on-chain execution, if off-chain methods execute any statements that write to the blockchain while executing off-chain, such writes have to be recorded and handed-off, together with transition outputs, for correctness attestation and subsequent recording on the blockchain.

Consequently, to “move a pattern off-chain”, we amend the smart contract to include special interface for the following three cases:

- (i) First invocation of the pattern in which appropriate data structures need to be prepared for the first and subsequent invocations of the off-chain methods.
- (ii) Subsequent invocations of the methods that are concerned with provision and return of appropriate parameters and data for methods invoked off-chain.
- (iii) Invocation of the pattern method that causes a transition out of the exit node, in which case, in addition to the returned parameters, the computation results produced by the off-chain computation that were saved in a cache are collected and are provided to the attestation procedure. Furthermore, once the results are approved, they are returned to the blockchain for recording (on the blockchain).

6 Tool for Off-chaining

We developed a tool, as a proof of concept, for the designer to:

1. Use HSM modeling of the smart contract application
2. Find patterns that are suitable for off-chain processing, wherein the patterns are identified by our algorithm to find simple subgraphs
3. Select a pattern for processing off-chain
4. Transform the HSM into a smart contract consisting of methods to be executed on the blockchain together with interface between on-chain and off-chain computation
5. Deploy the smart contract on the blockchain to interact with the off-chain computation.

The first three steps are independent from computing environments used for the blockchain and off-chain infrastructures. However, the last two steps are dependent. For the proof of concept, our tool targets the Ethereum blockchain with off-chain processing performed by sidechains, and we perform additional two steps:

6. Prepare and deploy the code for off-chain (sidechain in this case) pattern.

7. Execute the smart contract and display communication between the main chain and off-chain pattern.

It should be noted that as we execute the tool locally while running Ethereum on a local system, and that we use the Ganache tool to simulate the sidechain execution. As the pattern is executed on a sidechain that is also Ethereum based, interface between the Ethereum blockchain and the sidechain is much simplified because the sidechain has access to the blockchain state variables (data stored on blockchain) and the invocation of the sidechain methods are much simplified.

Although we only outlined the tool’s functionality, further details about the tool can be found in Appendix. Interested readers can actually test the tool by following the hyperlink: <https://chrisgagliu.github.io/FSM-model-application>. Keep in mind that due to cost consideration, the tool is running on a personal system that actually runs Ethereum platform on which the smart contract, with sidechain processing simulated by Ganache tool. As a consequence, the tool’s response time is slower than if it were deployed on a well-resourced cloud-based platform. Figure 3 shows the screenshot of the tool.

7 Summary

We addressed the issue of hybrid computing in which parts of a smart contract are computed off-chain. We decomposed the problem into three subproblems that can be broadly categorized into “What”, “When”, and “How”, respectively: (a) What is to be processed off-chain; (b) When processing off-chain should be facilitated; (c) How to facilitate automatically the interaction between on and off-chain processing.

FROM	TO	INPUT	TYPE	delete
INIT	SELLER_LIST_ITEM	seller list		Delete This Row
BUYER_MAKES_OF	SELLER_ACCEPT_C	buyer accept		Delete This Row
SELLER_ACCEPT_C	SELLER_SIGNED_C	seller signed		Delete This Row

Figure 3: FSM Model Specification

We described an algorithm we developed to find smart contract patterns that are suitable for to be processed off-chain based on a simple subgraph property. A simple subgraph of FSM states represents a pattern consisting of a collection of methods with

the following property if those simple subgraph methods are executed off-chain. Once a method, corresponding to the entry state is invoked, any further smart contract method invocations will be only to the methods of the pattern represented by the simple subgraph until the exit method is invoked, at which point any further invocations will be to methods executed on the blockchain, methods that are not a part of the simple subgraph. We created an algorithm to find such simple graphs and applied it on a Trade-finance use-case. To the best of our knowledge, this is the first attempt at identifying “what” parts of the smart contracts should be selected for processing off-chain. Previous research concentrates on the question of “how” to accomplish off-chain processing, i.e., they address how a given method, or a collection of methods, can be processed of chain. However, we do not claim our approach universally assists the developer in deciding whether a pattern should be executed off-chain across all blockchain use cases.

To ensure off-chain trustworthiness, we collect information on off-chain processing, that is input and output parameters of the smart contract methods and blockchain data read/written. The collected information is provided for attestation of results, produced by off-chain computing, as correct and for recording them on the blockchain together with attestation results. However, there is no efficient and universal approach for attestation and use semantic information may be required to facilitate efficient attestation methods.

To further support the decision on whether a pattern should be processed off-chain, we present to the designer detailed analysis of the overhead costs and benefits. These are obtained by transforming the HSM models into smart contracts, deploying them and measuring the actual costs and benefits.

We describe how we transform an HSM model into a smart contract that also includes interface, for the smart contract executing on the blockchain, to interact with the off-chain computation.

To validate our approach, we developed a tool that deploys the smart contract on Ethereum blockchain, while the patterns are processed on a sidechain. The tool enables a designer to create an FSM/HSM model and then use our algorithm to find patterns to be considered for processing off-chain. Once the patterns are found and the designer determines which patterns should be processed off-chain, she can use the tool to compile the smart contract and the patterns and deploy them. As the smart contract and the patterns are deployed, we can measure their cost with already existing tools and can thus provide to the designer information on the benefits and costs of processing off-chain.

REFERENCES

- [1] Asgaonkar A. & Krishnamachar B. (2019). Solving the Buyer and Seller's Dilemma: A Dual-Deposit Escrow Smart Contract. In IEEE Int. Conf. on Blockchain and Cryptocurrency (ICBC). 262-267.
- [2] Bodorik, P., Liu, C., Jutla, N. (2021) Using FSMs to Find Patterns for Off-Chain Computing. In Proceedings of the 3rd International Conference on Blockchain Technology (ICBCT '21). IEEE (in press).
- [3] Eberhardt J., Jonathan H. (2018) Off-chaining Models and Approaches to Off-chain Computations. In Proceedings of the 2nd Workshop on Scalable and Resilient Infrastructures for Distributed Ledgers (SERIAL'18). 7-12.
- [4] HeartBank (2018). Smart Contract Design Patterns: A Case Study with Real Solidity Code. Retrieved from <https://medium.com/heartbankstudio/smart-contract-design-patterns-8b7ca8b80dfb> on Nov. 2, 2020.
- [5] Lisk. (2020). What is Blockchain?. Retrieved from <https://lisk.io/what-is-blockchain> on Nov 2, 2020.
- [6] Liu, C. (2021). FSM Modeling for Off-blockchain Computation. MCSC Thesis, Dalhousie University.
- [7] Ledger Insights. (2018). Letter of Credit blockchain launches with eight banks. Retrieved from <https://www.ledgerinsights.com/blockchain-letter-of-credit-trade-finance-voltron/> on Nov 8, 2020.
- [8] Mavridou, A., & Laszka, A. (2018(a)). Designing Secure Ethereum Smart Contracts: A Finite State Machine Based Approach. Financial Cryptography. DOI: 10.1007/978-3-662-58387-6_28
- [9] Mavridou A., Laszka A. (2018(b)) Tool Demonstration: FSolidM for Designing Secure Ethereum Smart Contracts. In: Bauer L., Kesters R. (eds) Principles of Security and Trust. POST 2018. Lecture Notes in Computer Science, vol. 10804. Springer. 270-277.
- [10] Poon, J. & Buterin V. (2017). Plasma : Scalable Autonomous Smart Contracts. <https://plasma.io/plasma.pdf> on Nov. 12, 2020. 47 pages.
- [11] Yannakakis M. (2000) Hierarchical State Machines. In: van Leeuwen J., Watanabe O., Hagiya M., Mosses P.D., Ito T. (eds) Theoretical Computer Science: Exploring New Frontiers of Theoretical Informatics. TCS 2000. Lecture Notes in CS, vol 1872. Springer. 315-330.

Appendix

Here we show further details about the tool we developed to support moving a pattern of a smart contract for processing off-chain. Although we already showed a figure of the tool and briefly described it in the main part of this paper, we repeat the that figure here, as Figure A-1, for completeness and further description. All figures are for the use case already described in Section 4 of the main body of this paper, use case that involves a seller and a buyer of some product that includes an escrow deposit.

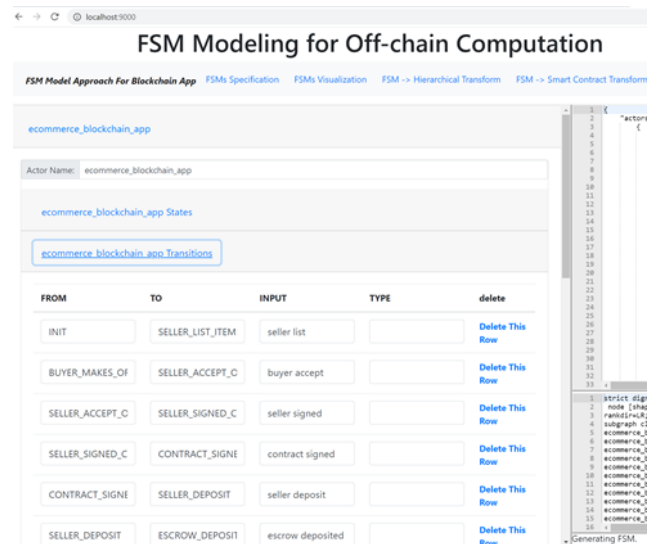


Figure A-1: FSM Model Specification

Below the “FSM Modeling...” title is a bar containing buttons used to proceed from one step to another. The screenshot is taken after pressing the “FSMs Specification” button and shows definition of transitions between. Not all transitions of the use case are shown in the figure.

Once the FSM is created by the designer, it can be visualized (using the “FSM Visualization” button). The result is shown in Figure A-2, which displays a portion of the resulting FSM graph. The user can scroll in both directions when viewing the graph as well as change size.

The next step in modelling shows the result of finding patterns for off-chain consideration by applying our algorithm to find simple subgraphs (after pressing the “FSM->Hierarchical” Transform button), the result of which is shown in Figure A-3. It shows some of the patterns that were identified. In reference to Figure 1 in the main body of this paper, it shows patterns S6 through S9.

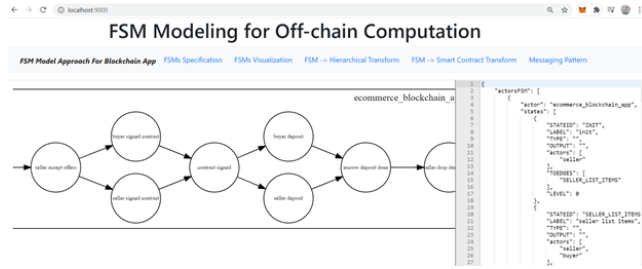


Figure A-2: FSM Visualization

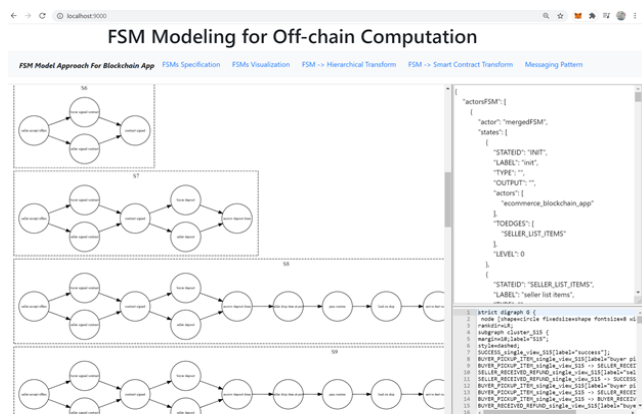


Figure A-3: Found Patterns

The subsequent steps deal with transformation of the HSM model into a smart contract to be executed on the Ethereum blockchain. In addition, an interface is included in the smart contract for interacting with the pattern that is processed off-chain. As the sidechain execution proceeds on an Ethereum sidechain, we also deployed the pattern automatically on the sidechain. For interaction and signaling between on-chain and side-chain computation by the bridge process, we use IPFS Message channels. IPFS is the InterPlanetary Files System protocol for sharing data in a distributed file system.

Figure A-4 shows the result of transforming the FSM into HSM and being ready for compilation and deployment. The user is able to compile the selected patterns for off-chain and to compile the smart contract for the blockchain. The top-left-hand side of

the panel is for compilation while the top-right-hand side for deployment.

Once patterns and the main smart contracts are deployed (Figure A-5), their execution is monitored by showing messages that are cause an entry into a pattern execution. Figure A-6 shows a screenshot of the messages from the on-chain off-chain interface using IPFS. The main smart contract is instrumented to repeatedly start in the initial state and proceed to the final state.

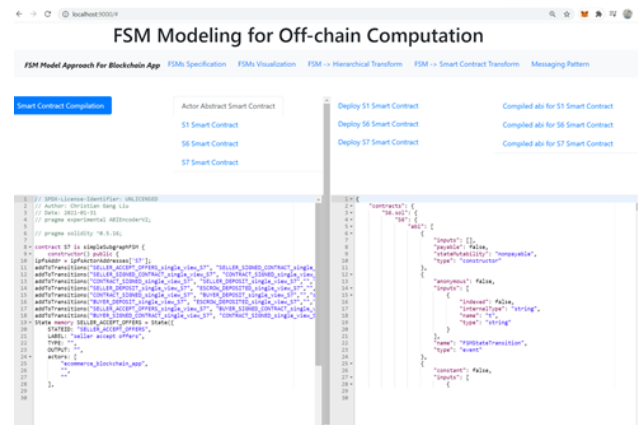


Figure A-4: Smart Contract Compilation

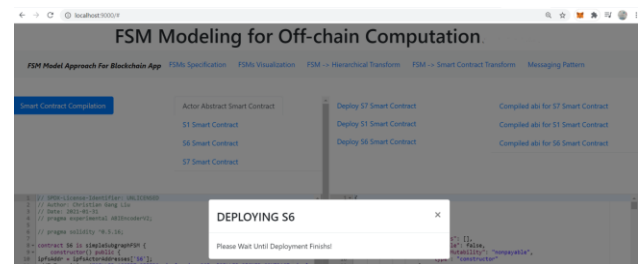


Figure A-5: Deployment of the Pattern S6

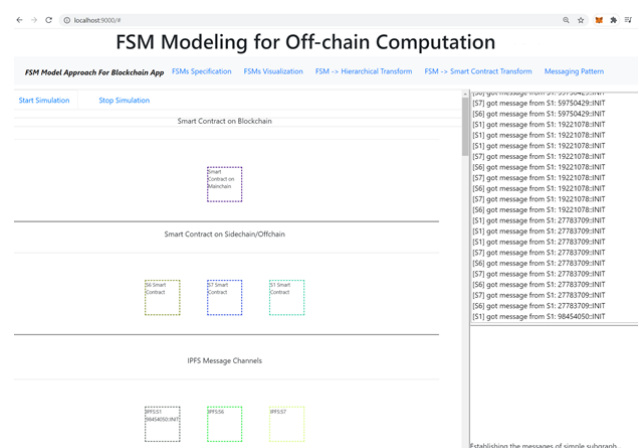


Figure A-6: Messages showing repeated pattern execution